

LABORATORY OF AUTOMATION SYSTEMS

Davide Samorì

Dipartimento di Ingegneria dell'Energia Elettrica e dell'Informazione (DEI)

Università di Bologna

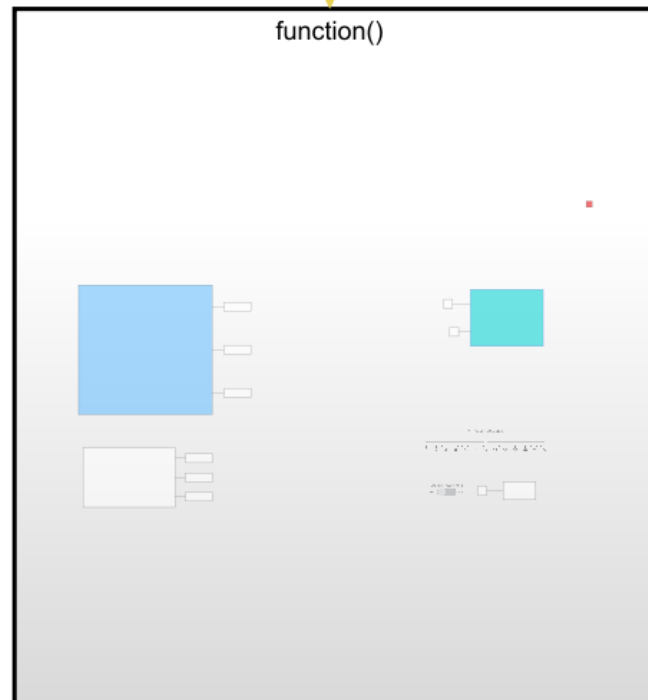
Email: davide.samori@unibo.it

You can download and open:

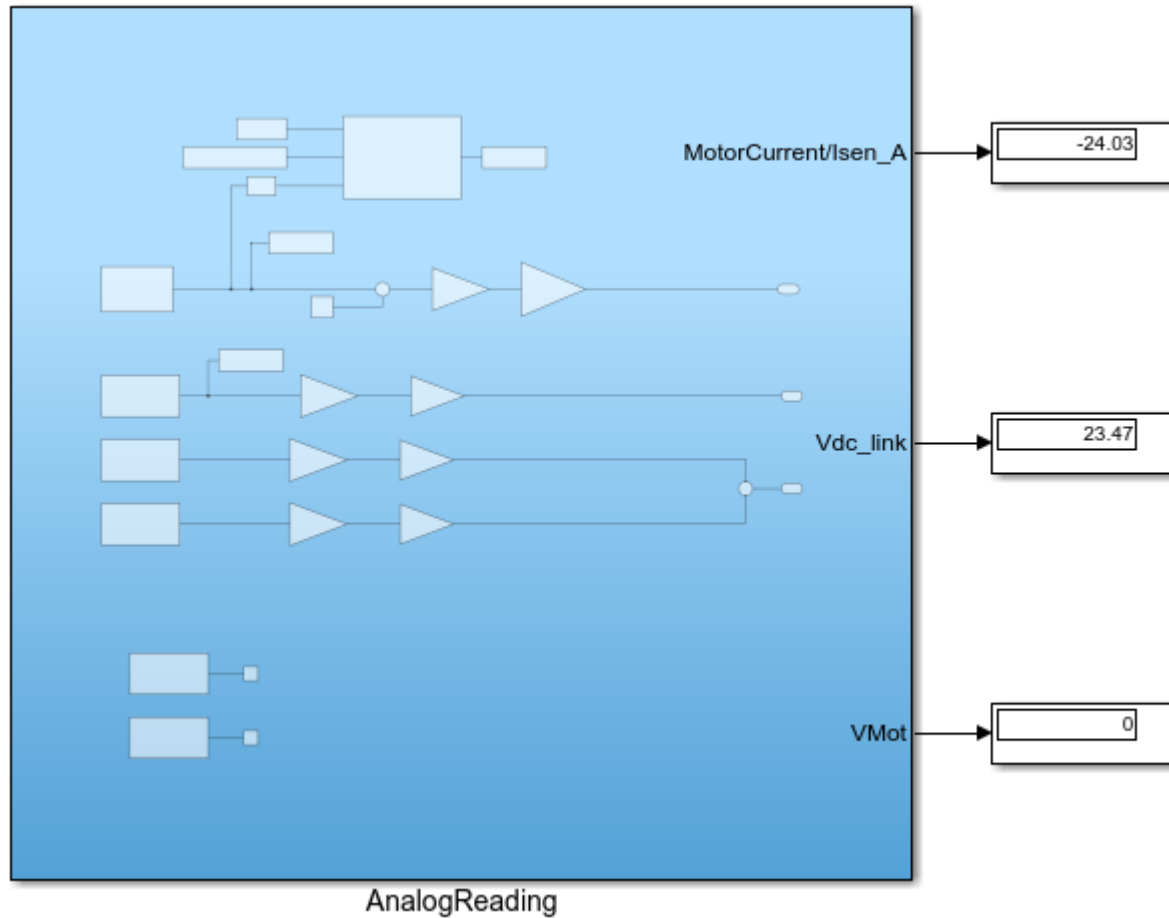
- BaseModelForControl.slx
- BaseModelForControlParameters.m
- Your DC Motor Model

Today Goals:

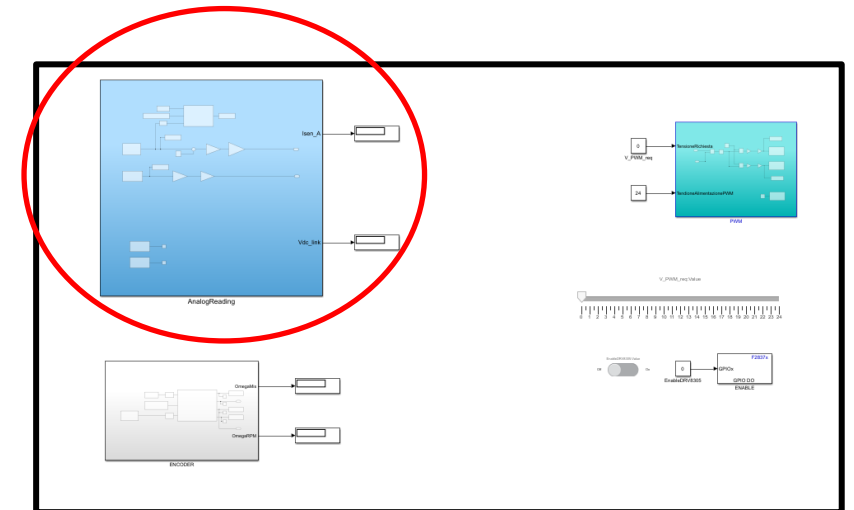
- Have a look of our first interface
- Check measure (I, V, Omega, ecc...)
- Some optimization
- Start PI design

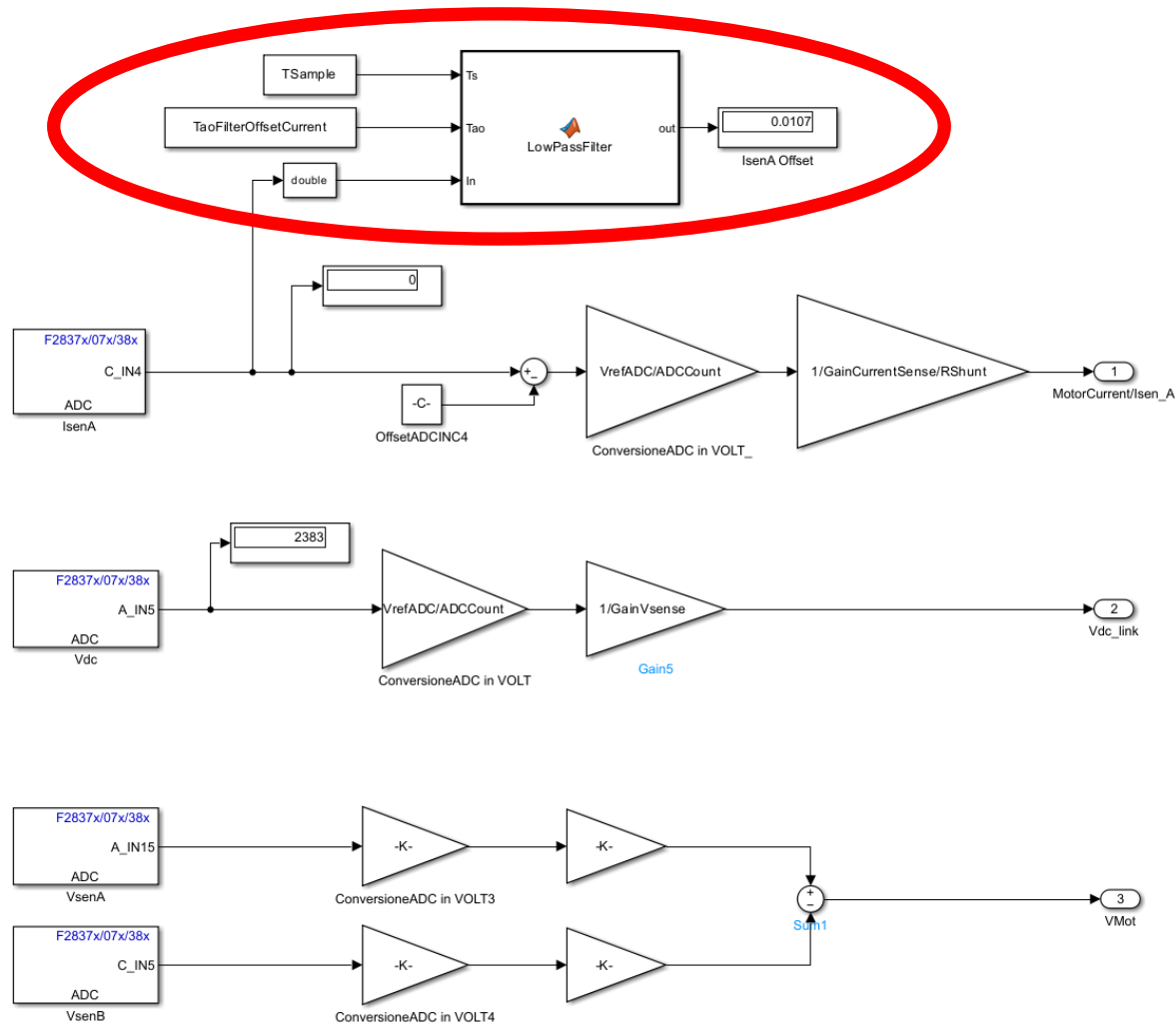


This is our main task.
Every TSample=100us we run it



Inside we have a block for analog reading

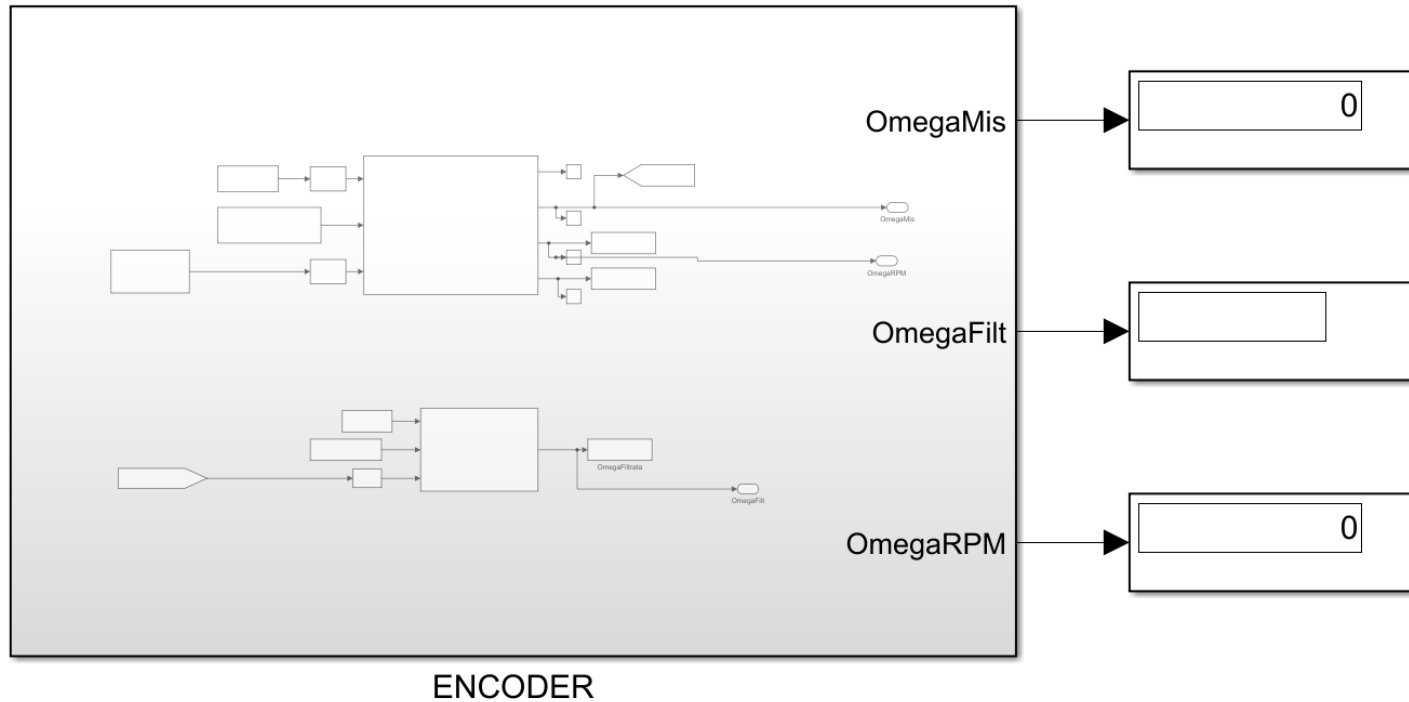




Motor Current

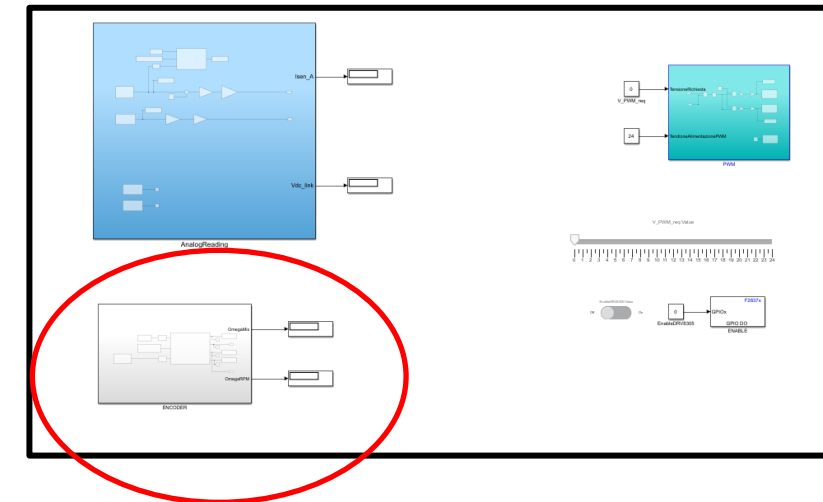
VDC_Link / PowerSupply

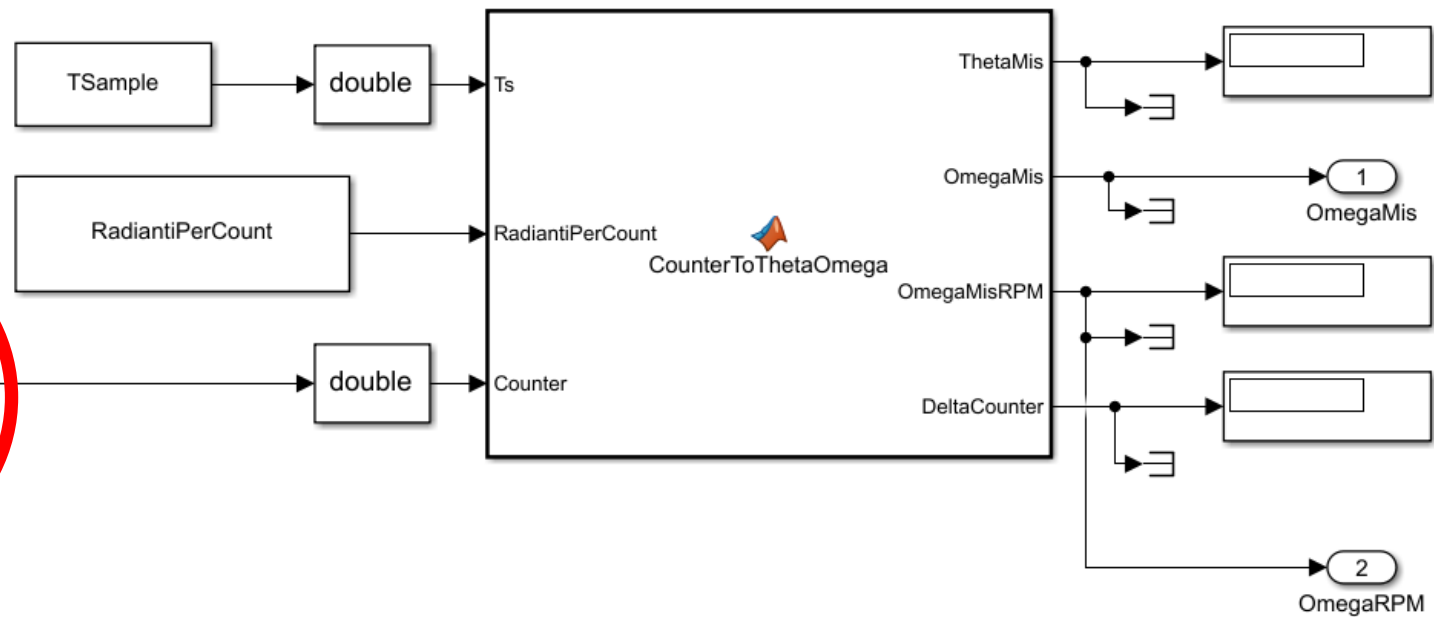
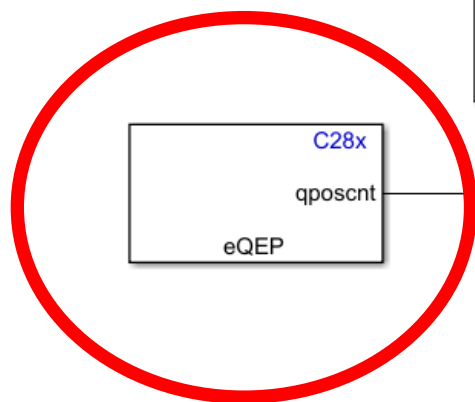
Motor Voltage

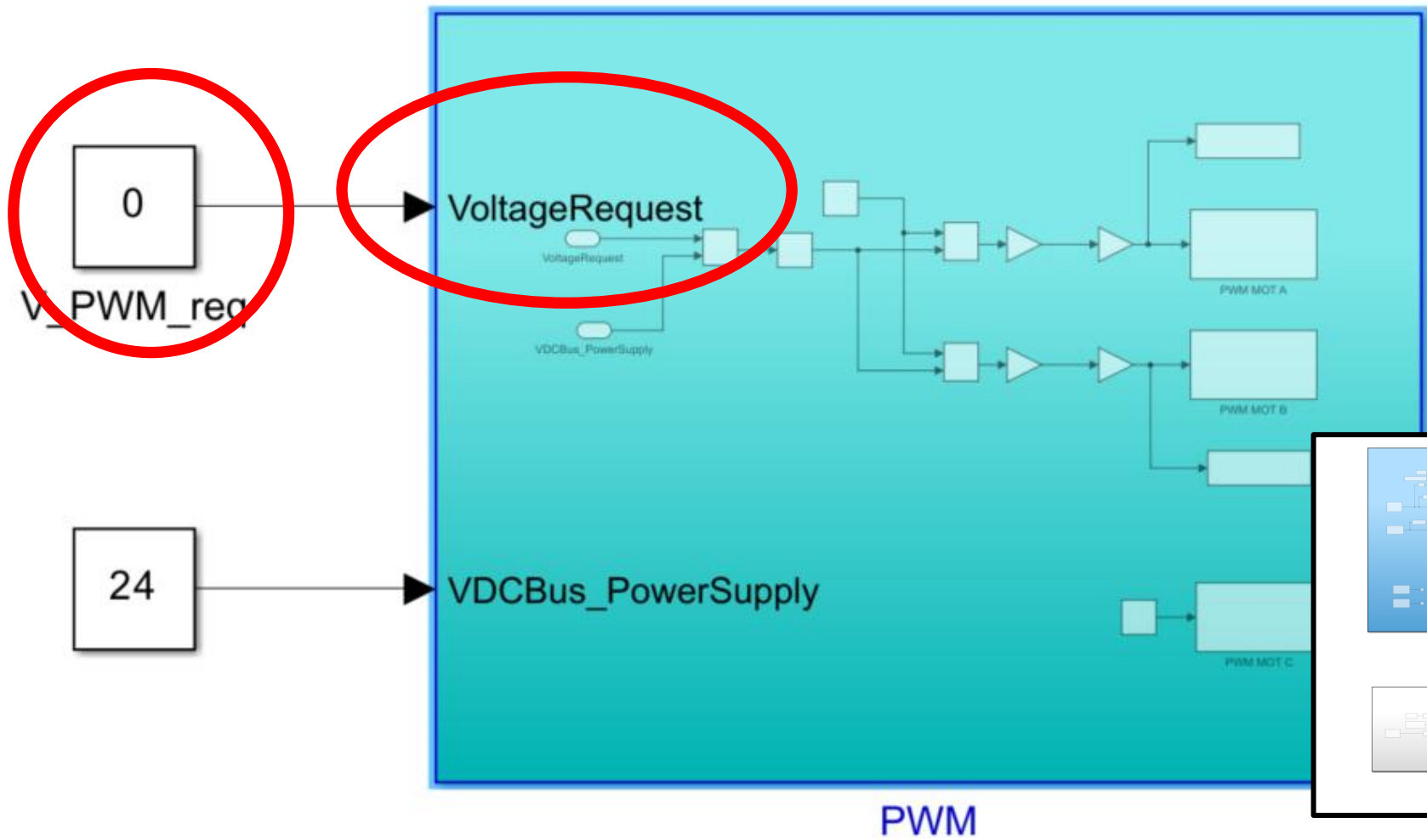


Encoder Block:

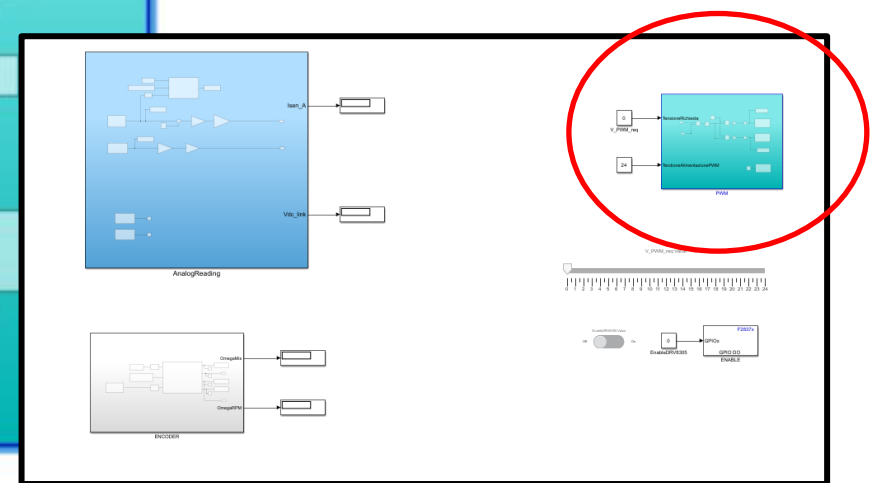
- Ω_{Mis} (rad/s) S.I.
- Ω_{Filt} (rad/s) S.I.
- Ω_{RPM} (rpm)

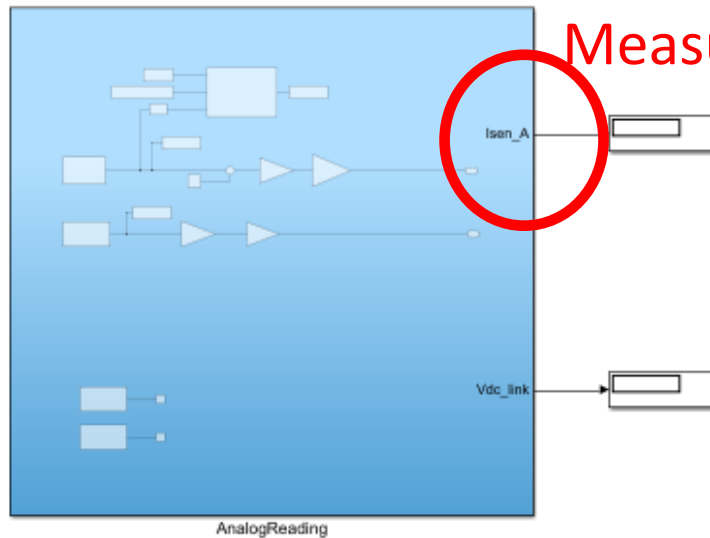






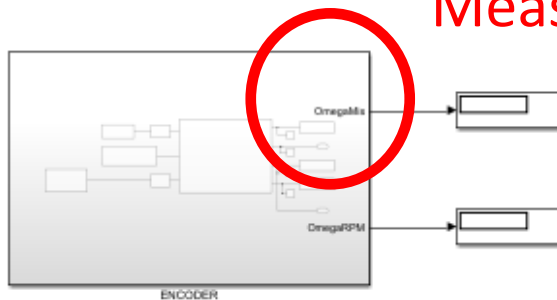
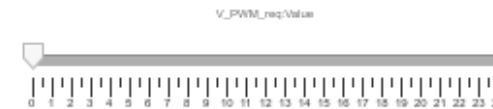
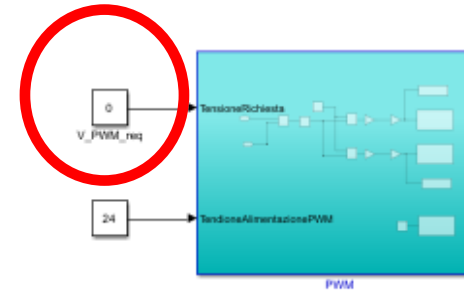
Here we can ask for a voltage: a value between -24V e +24V





Measured Current

Voltage to the motor



Measure speed (rad/s)



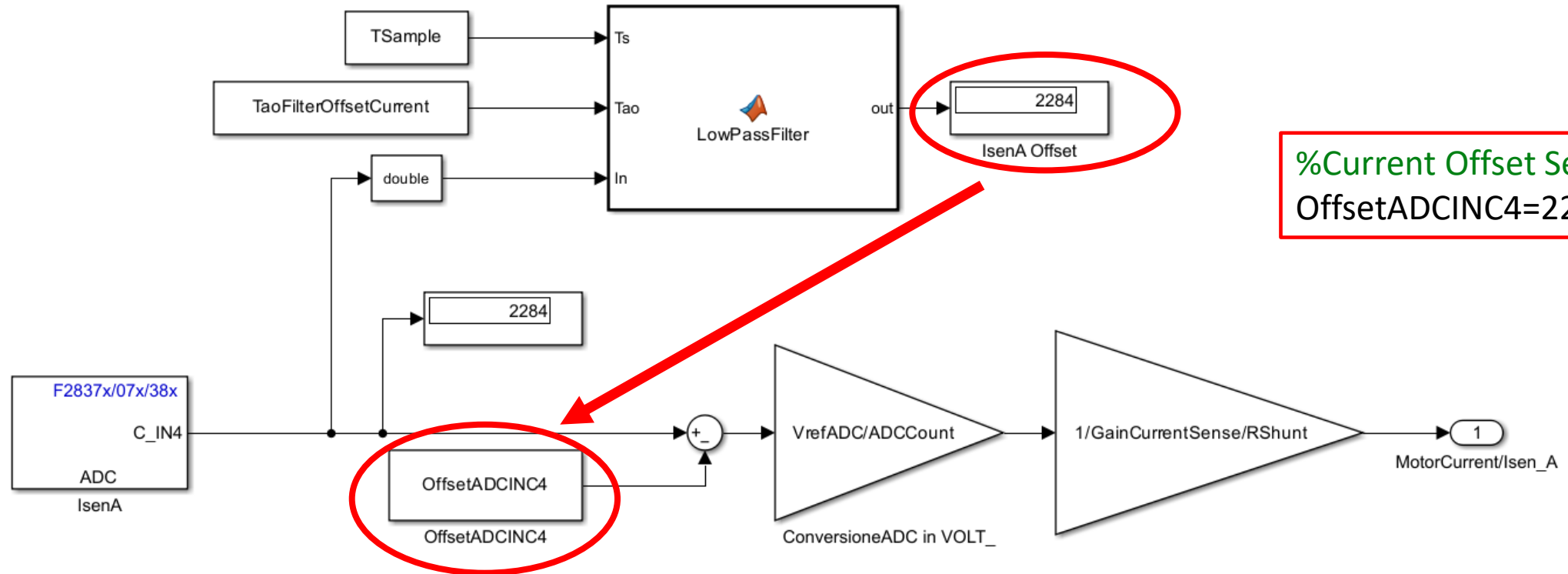
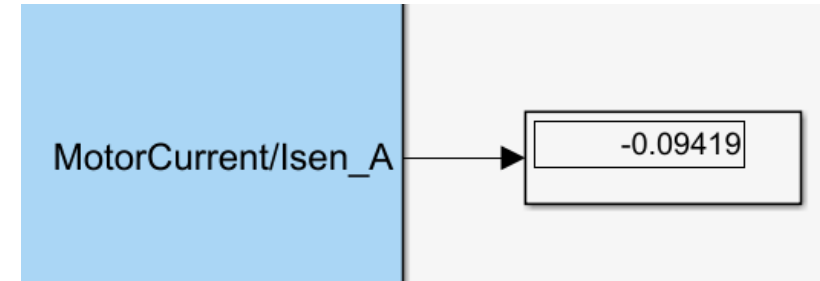
We don't have motor model, we use real motor

You can Run Model:

- Run the motor
- Watch Speed, Current, ecc...
- Current with/without Load

© 2011 Pearson Education, Inc. or its affiliate(s). All rights reserved. This material is intended solely for the personal use of the individual user and is not to be disseminated broadly.

Voltage Request = 0V



%Current Offset Setup #2

```
OffsetADCINC4=2284;
```

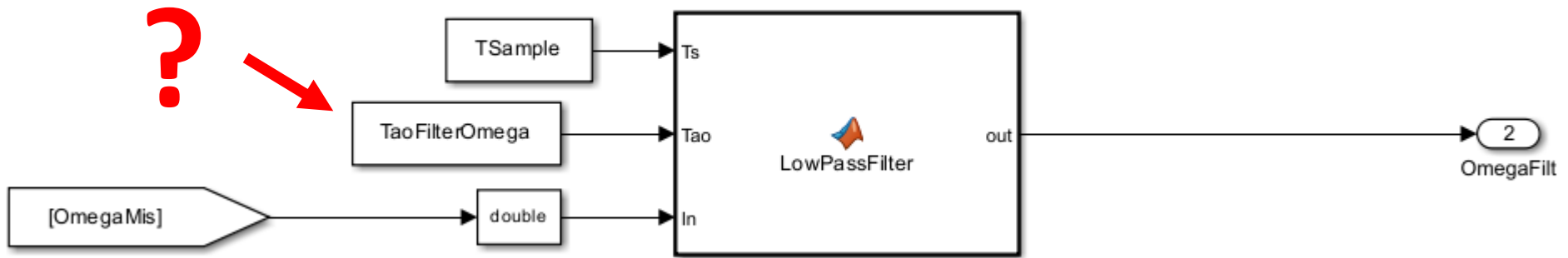
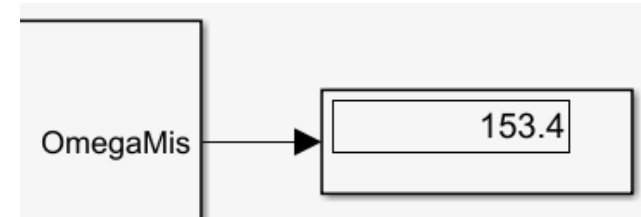
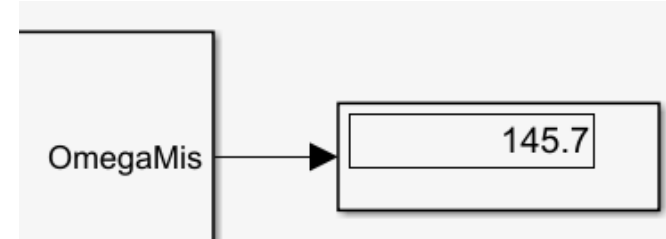
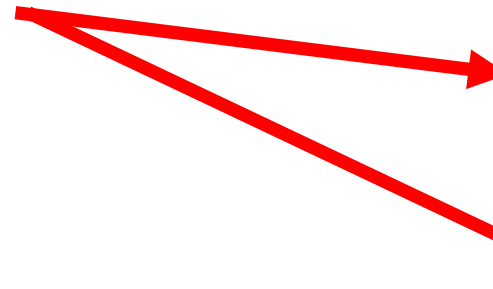
FILTER

Enable Drive = 1

Voltage Request = 24V

% Omega filter - To Be Defined

TaoFilterOmega=.....;



FF

Enable Drive = 1
Voltage Request = 12V



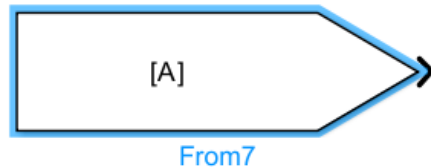
Change Power Supply Voltage
24V vs 15V



Look Vmotor and speed



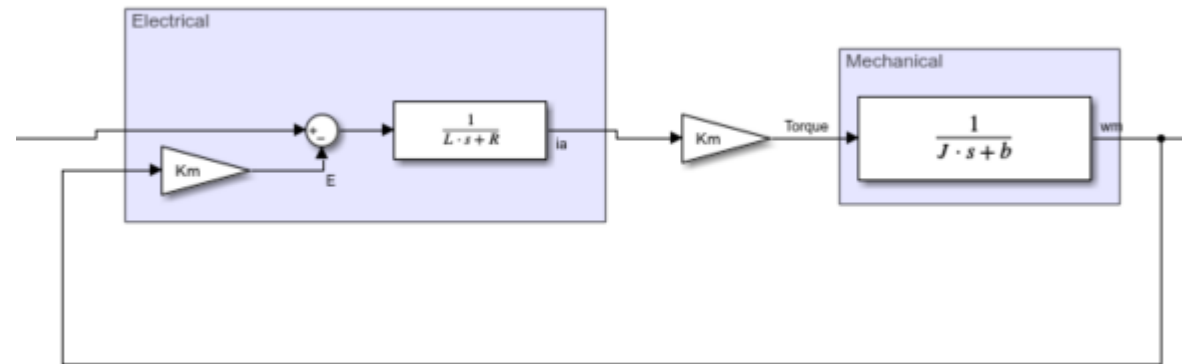
How can we avoid it quickly?



OK, now our interface runs well....

NOW, we want to start controlling our motor

Please open your DC Motor Model



Model of a DC electric motor

Note that the transfer function

$$G_{\omega}(s) = \frac{k_m}{L_A J s^2 + (R_A J + b L_A) s + (b R_A + k_m^2)}$$

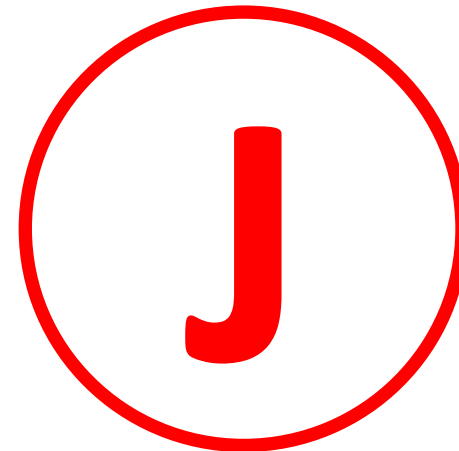
%% TRANSFER FUNCTION

s=tf('s');

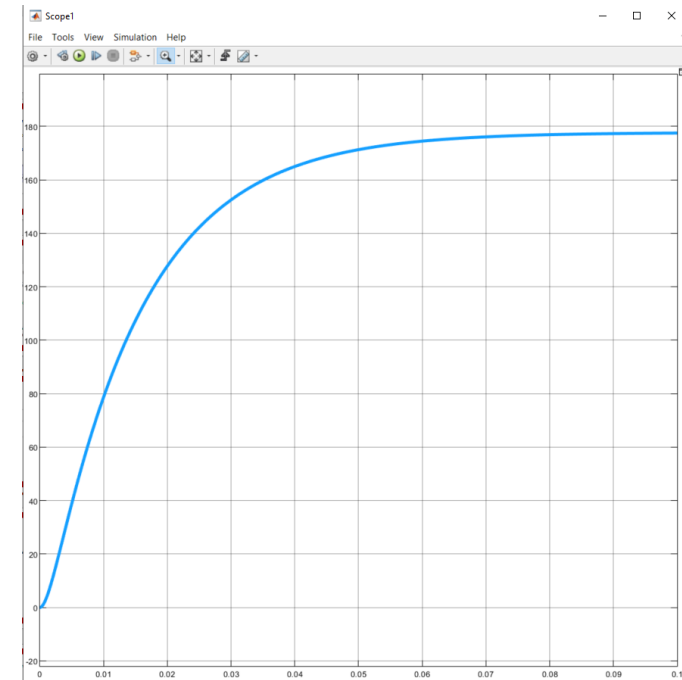
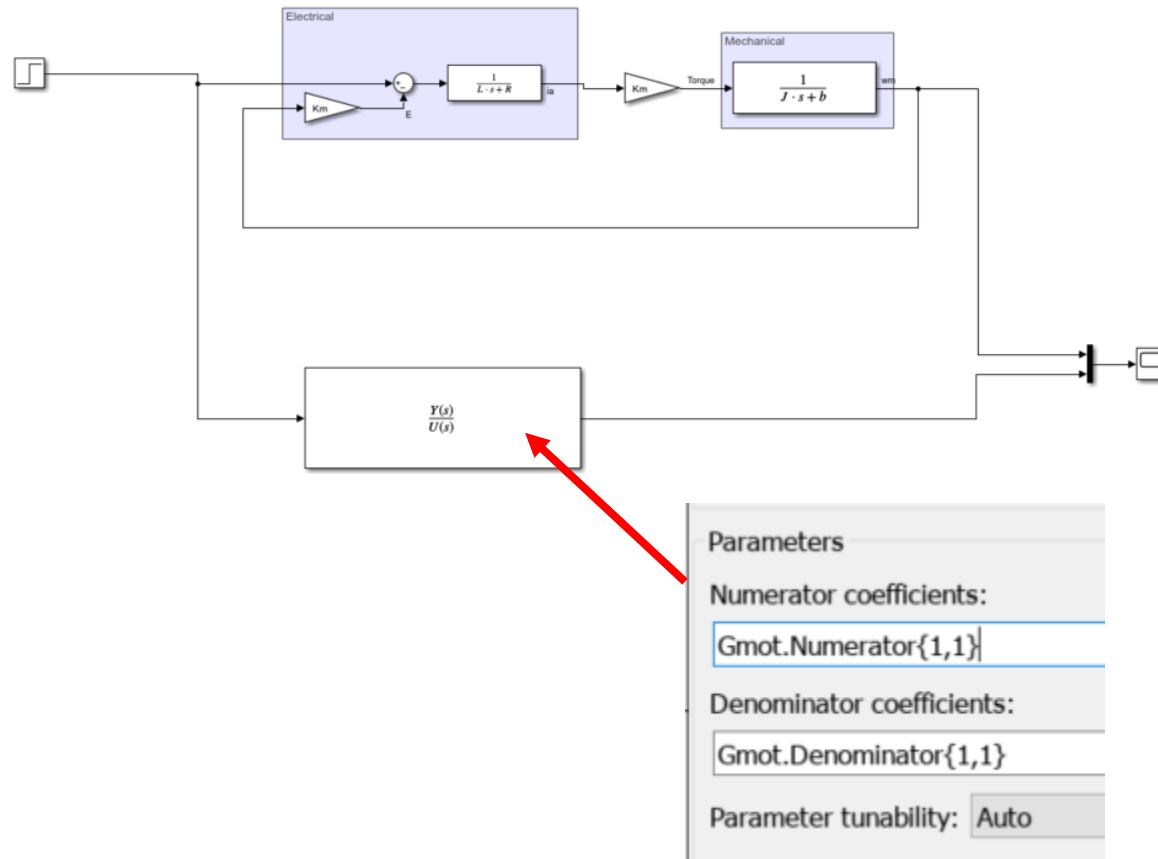
z=tf('z',TSample);

%DC Motor continuous transfer function

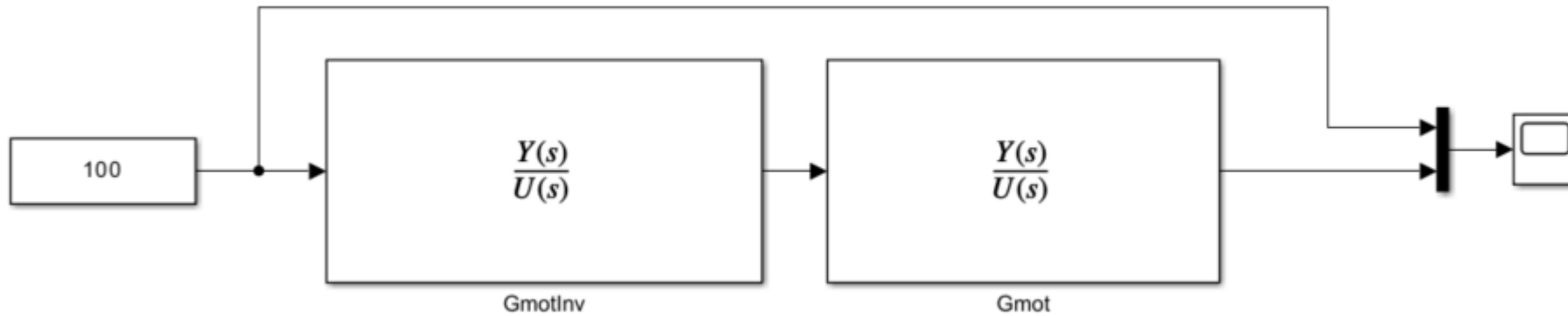
Gmot=tf([Km],[L*J R*J+b*L Km*Km+b*R])



A quick check...



$G_{motInv}=1/G_{mot}$



% I need two poles

pole1=1/(1+0.002*s);

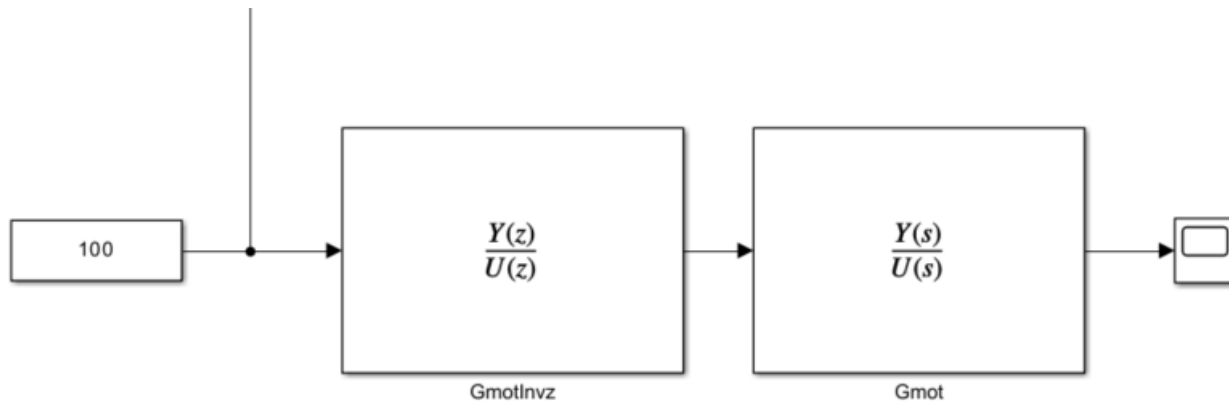
pole2=1/(1+0.001*s);

$G_{motInv}=G_{motInv}*pole1*pole1$

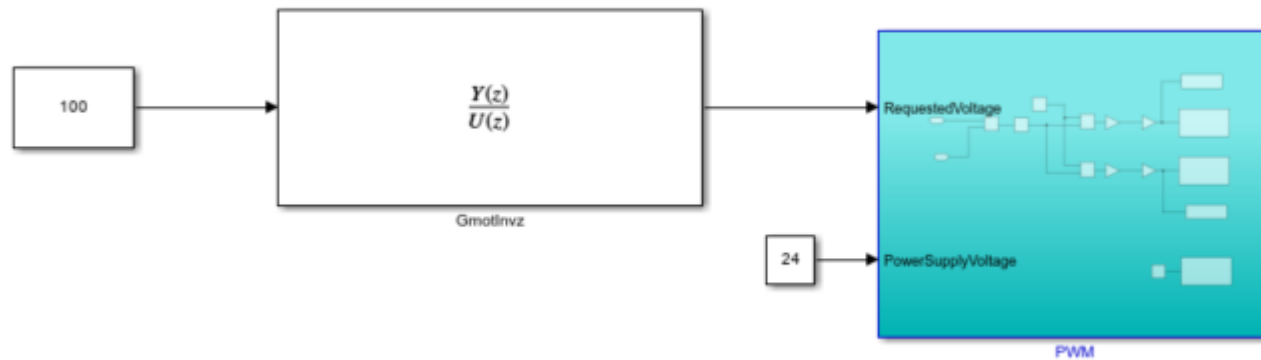


% GmotInv Discretization

GmotInvz=c2d(GmotInv,TSample)



You can copy this block to BaseModelForControl

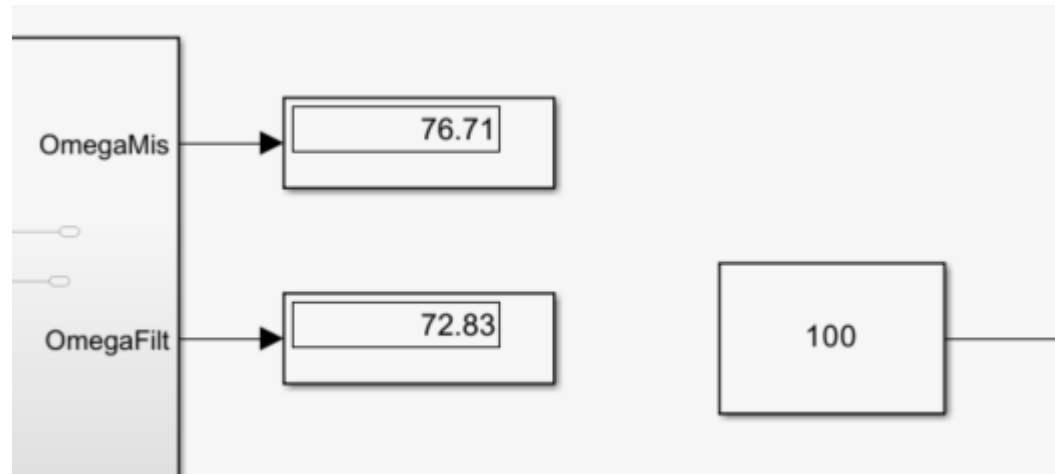


Did you have good result?



Try with brakes resistors...

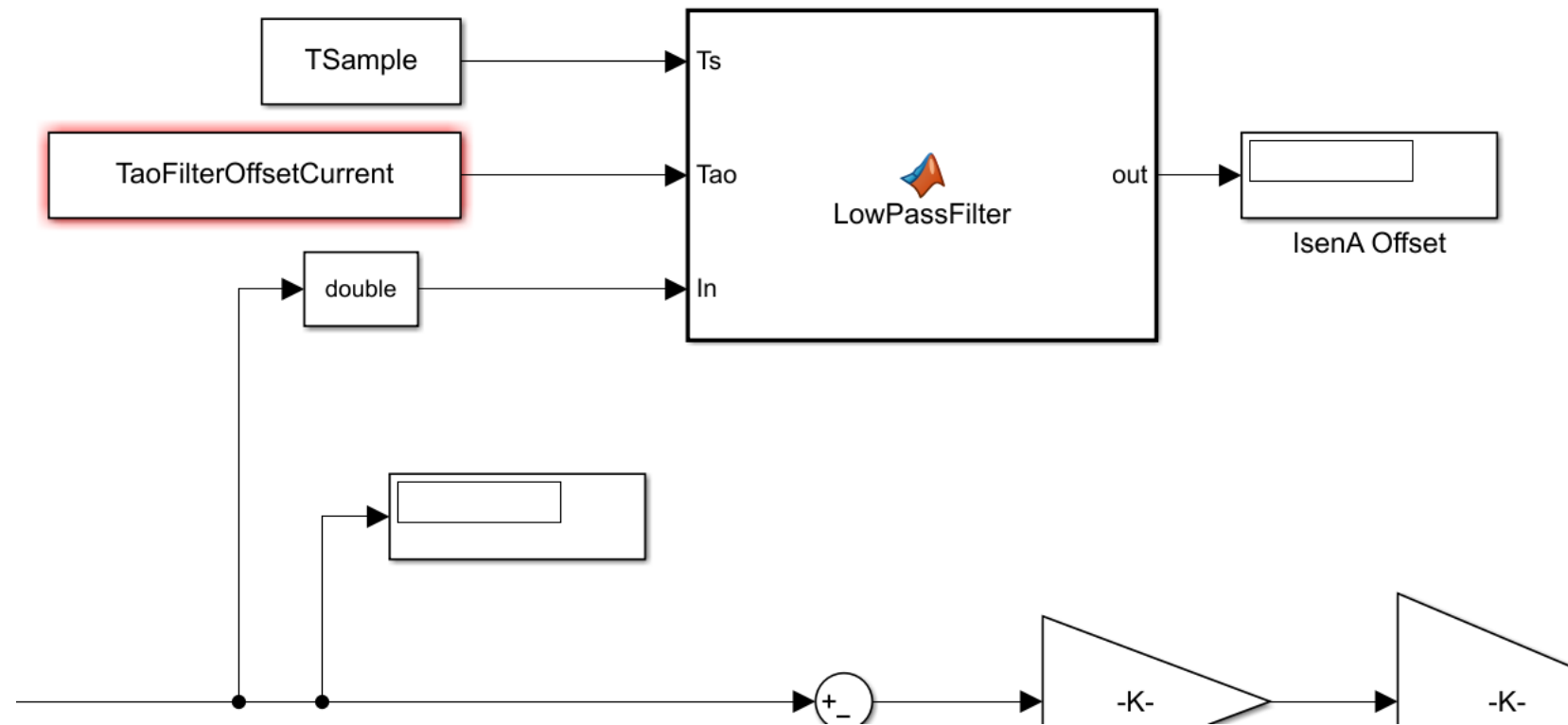
What's happen?



Try with **$b = J/Taom$**

A quick review about the filter

We realized a discrete time filter



EXAMPLE: FILTER BACKWORD DIFFERENCE DISCRETIZATION

$$G(s) = \frac{1}{1 + s\tau} \quad \left| \quad s = \frac{z-1}{Tz} = \frac{1-z^{-1}}{T} \right.$$

$$G(z) = \frac{1}{1 + \frac{1-z^{-1}}{T}\tau} = \frac{Y}{X} \quad \longrightarrow \quad Y \left(1 + \frac{\tau}{T} - \frac{\tau}{T} z^{-1} \right) = X \quad \longrightarrow \quad Y \left(1 + \frac{\tau}{T} \right) = X + Y \frac{\tau}{T} z^{-1}$$

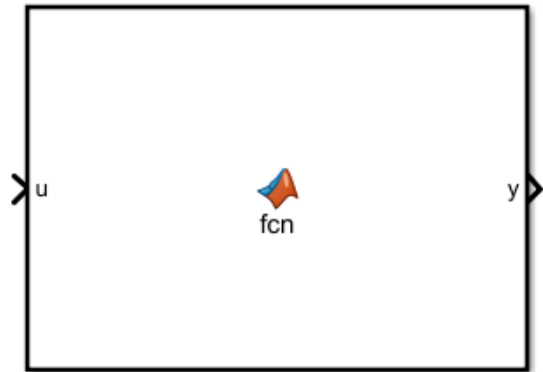
$$Y = \frac{T}{T + \tau} X + \frac{\tau}{T + \tau} Y z^{-1}$$

$\longrightarrow$

$$Y = \frac{T}{T + \tau} X + \frac{\tau}{T + \tau} Y_{\text{prec}}$$

T=tempo di campionamento

Matlab Function



Doppio click

```
function y = fcn(u)
```

```
y = u;
```

Prendiamo una Matlab Function



$$Y = \frac{T}{T+\tau} X + \frac{\tau}{T+\tau} Y_{\text{prec}}$$

```
function out = LowPassFilter( Ts ,Tao, In)

persistent OutPrev
if isempty(OutPrev )
    OutPrev = 0 ;
end

% Low-Pass filter
% discretized by backward difference

a=Ts/ (Ts+Tao) ;
b=Tao/ (Ts+Tao) .

out=a*In + OutPrev *b;

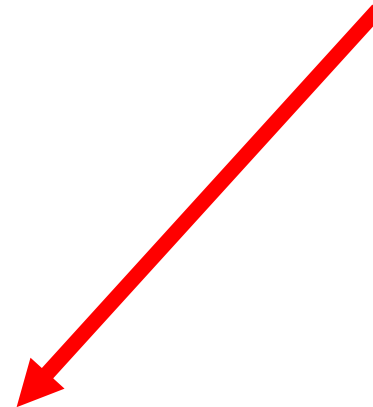
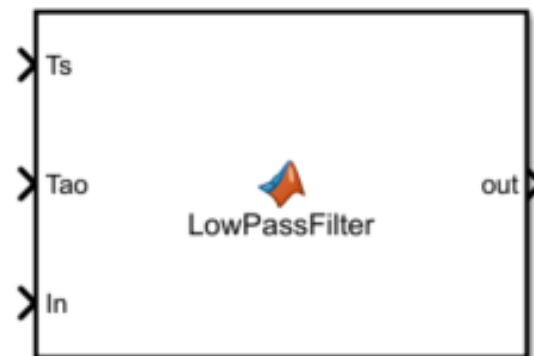
OutPrev = out;

end
```

$$G(s) = \frac{1}{1 + s\tau} \quad \left| \quad s = \frac{1 - z^{-1}}{T} \right.$$



$$Y = \frac{T}{T + \tau} X + \frac{\tau}{T + \tau} Y_{\text{prec}}$$



T=tempo di campionamento



PI !!

T=tempo di campionamento

PI: TUSTIN DISCRETIZATION

YOU CAN TRY WITH DIFFERENT DISCRETIZATIN TECNIQUES

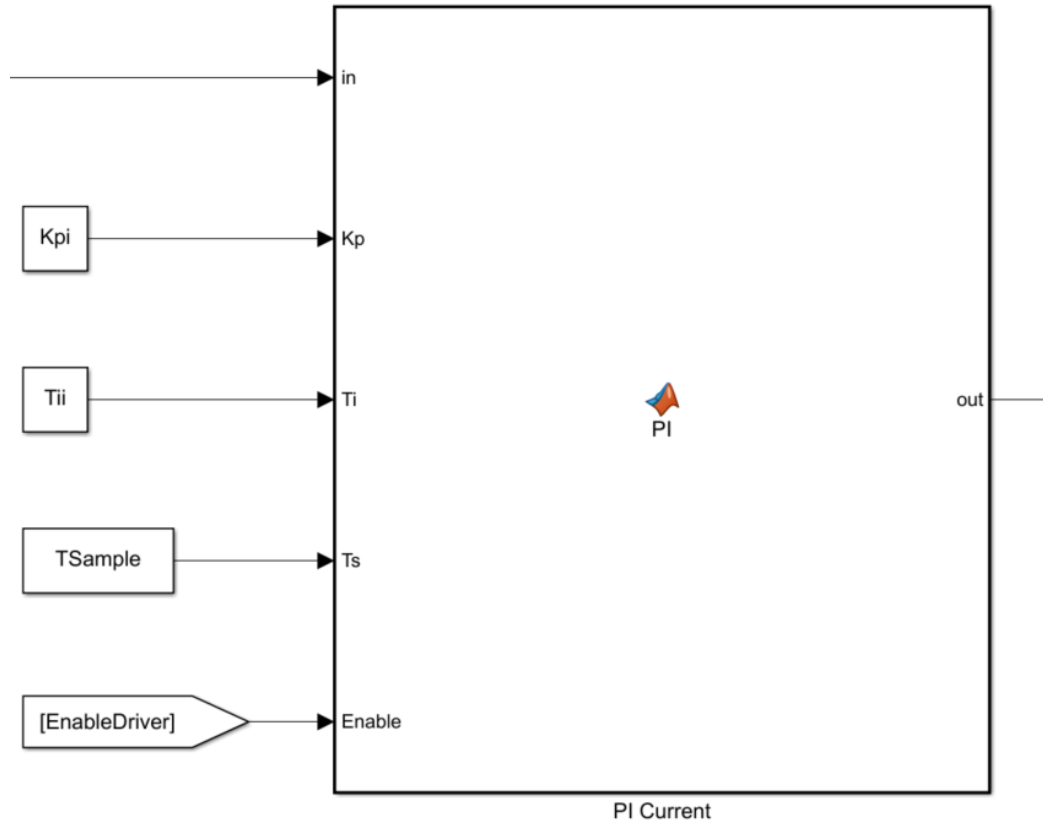
$$R(s) = K_p \cdot \left(1 + \frac{1}{T_i \cdot s}\right) \quad \left| \quad s = \frac{2}{T_s} \frac{1 - z^{-1}}{1 + z^{-1}} \right.$$

$$R(z) = K_p \cdot \left(1 + \frac{1}{T_i \cdot \frac{2}{T_s} \frac{1 - z^{-1}}{1 + z^{-1}}}\right) = \frac{Y}{X} \quad \longrightarrow \quad \dots \dots \dots$$



$$Y = K_p \cdot (X - X_{\text{prec}}) + \frac{K_p \cdot T_s}{2 \cdot T_i} \cdot (X + X_{\text{prec}}) + Y_{\text{prec}}$$

T=tempo di campionamento



```

function out = PI( in ,Kp , Ti , Ts , Enable)

persistent in_prev out_prev

% Initial Value
if isempty( out_prev )
    out_prev = 0;
end

if isempty( in_prev )
    in_prev = 0;
end

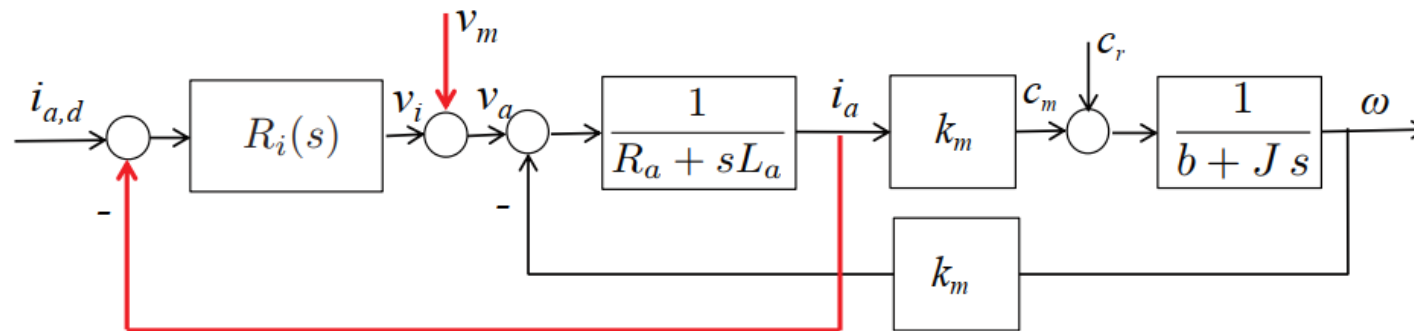
%  $G(s) = (Kp/Ti) * ((1+Ti*s)/s)$  (Tustin)
% We run control only if motor is running
if (Enable == 1) ←
    %I Calculate the output
    out = Kp*Ts/(2*Ti)*(in+in_prev) + Kp*(in-in_prev) + out_prev;

else
    out = 0;
    in = 0;
end
out_prev = out;
in_prev = in;
end

```

- Connect it in our scheme, input, output, error signal, ecc...
- Watch Prof. Melchiorri Slide (Design of Motion Control System) from pag. 24
 - Tii
 - Kpi
 -

The classical block scheme for the current control loop is



- Add a display on PI Output
- Add a display on error signal
- Connect Brake Resistor
- Start with a SetPoint of 0.1A
- Increase with a step of 0.1 (0.2, 0.3, 0.4,) until the motor runs.
- Check if error signal is “Low”

DISCONNECT BRAKE RESISTOR

- Start with a SetPoint of 0.1A
- Increase with a step of 0.1 (0.2, 0.3, 0.4,) until the motor runs.
- Apply a SetPoint of 0
- Apply a Set point of 1A
- Wait some seconds
- Apply a SetPoint of 0

- **Did the engine stop?**
- **Why?**
- Watch what's happen to voltage

SATURATION

- In Prof. Melchiorri slide you can find different anti windup technics, for the next time have a look and prepare one solution **HOMEWORK**
- Now we can quickly apply a fast improvement but not perfect.
 - After calculating the output we saturate the value

```
out = .....;

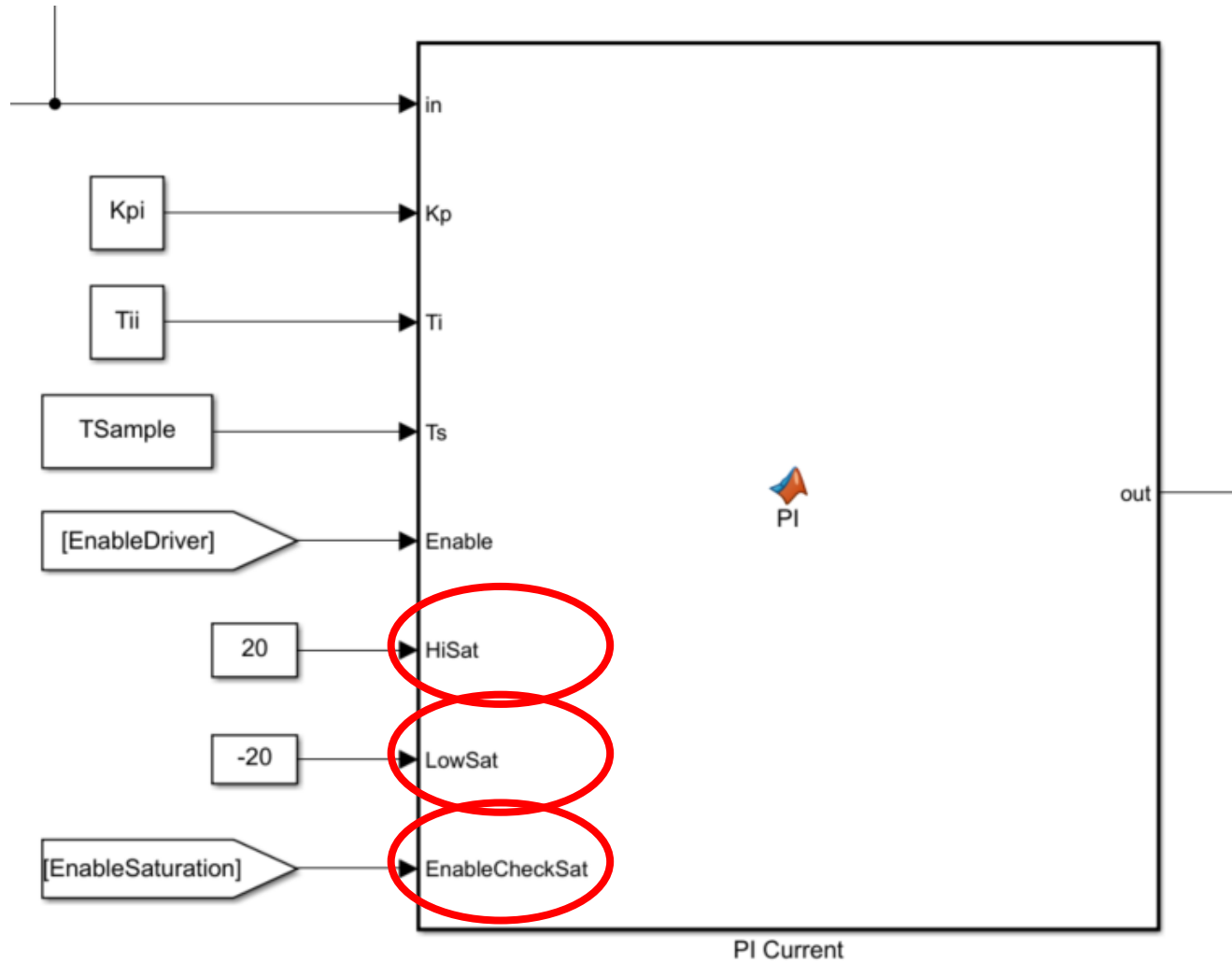
if(EnableCheckSat == 1)

    %check for saturation
    if out > HiSat
        out = HiSat;

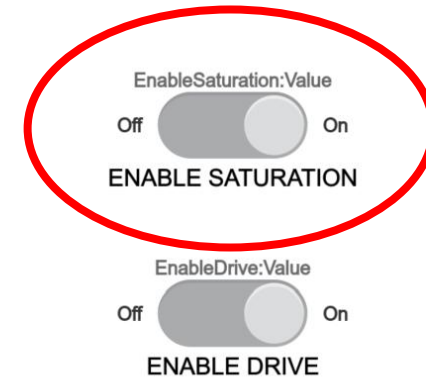
        elseif (out < LowSat)
            out = LowSat;

    end
end
```

Difference with c2d()

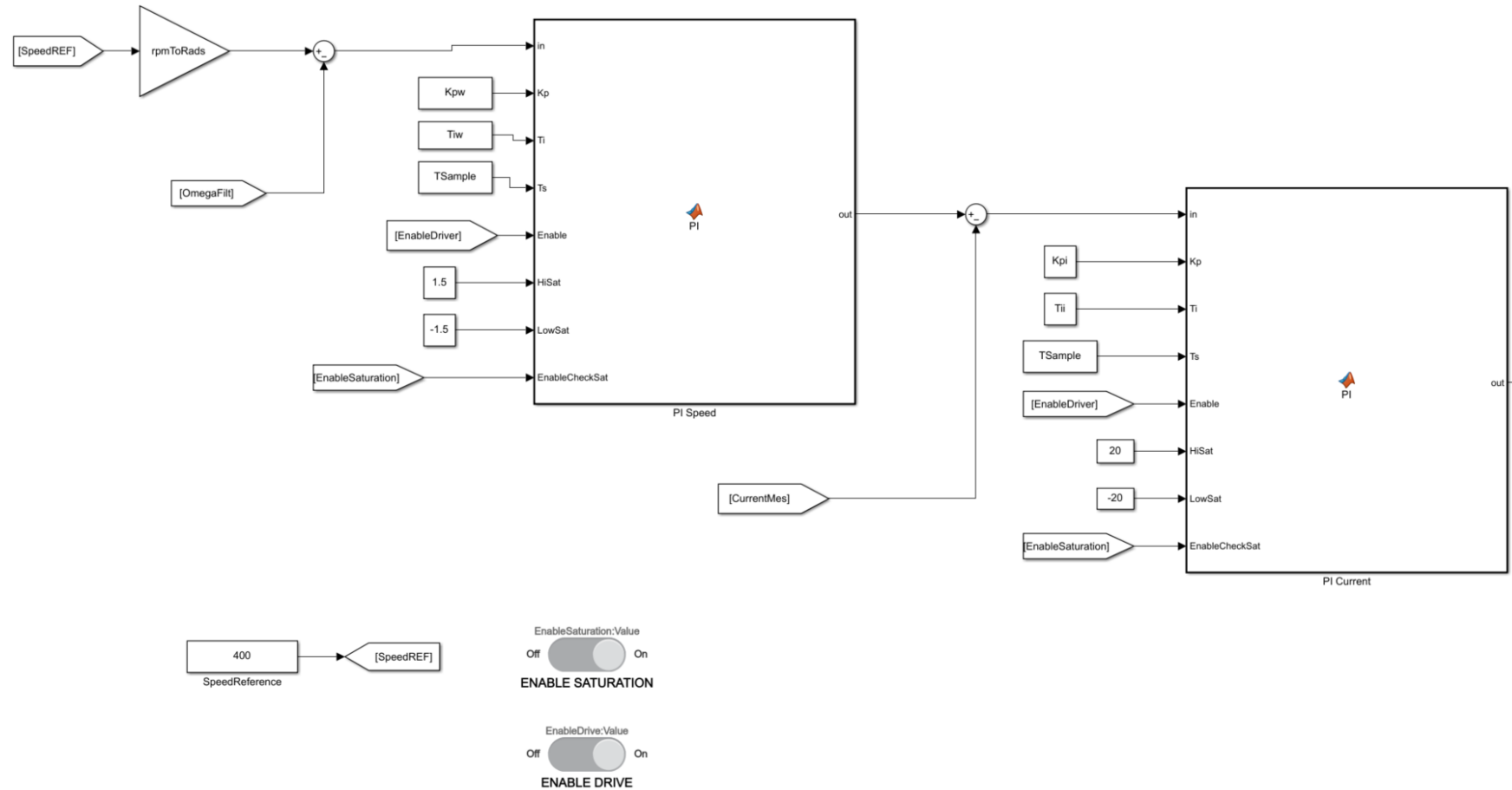


Add another Slider Switch

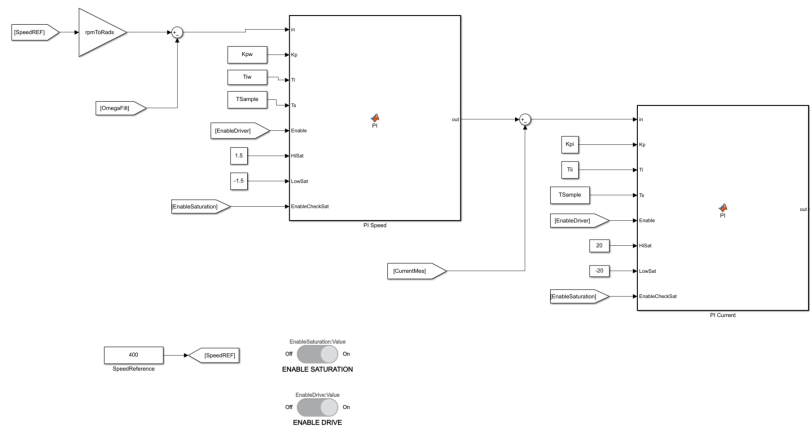


**Try what's happen now
with and without
EnableSaturation**

Now we can close also the speed loop



- pay attention to the unit of measurement of omega
- Try with or without load (watch voltage,...)
- What's about saturation value for Speed Loop?



How can I test my control with motor model?